Hafta 5 — Derste Yapılacak Uygulamalar

> **Toplam süre:** 33 dakika — blok sürelerinin **içindedir** (bkz. `H05-anlatim.md` bölüm 2)
>
> Bu haftanın bütün uygulamaları **Ctrl+U** etrafında döner. Anlatımın merkezi yazdığımız Razor değil, **tarayıcıya giden HTML**.

🔧 Hazırlık — Derse girmeden önce (5 dk, ders dışı)

- [ ] `Hafta-05/KampusKitap` açıldı, `dotnet run` ile çalıştığı doğrulandı
- [ ] `/Etiket`, `/Etiket/Baglanti`, `/Etiket/Csrf`, `/Etiket/Eksik` açıldı
- [ ] `/Kitap/Ekle` ve `/Kitap/EkleTH` **iki sekmede yan yana** açıldı
- [ ] İki sekmede de **Ctrl+U** denendi; kaynak penceresi yazı tipi **büyütüldü**
- [ ] `Program.cs` editörde açık ve rota satırı **görünür durumda** (Uygulama A'da değiştirilecek)
- [ ] Tarayıcı yakınlaştırması **125–150%**
- [ ] `H05-etkilesimli.html` açıldı; N denendi ve **kapatıldı**
- [ ] Slayt 18'deki 3B sahnenin açıldığı görüldü (ağ gerekiyor; önceden bir kez açın)

> 💡 **Zaman kazancı:** Bu haftanın bütün sayfaları projede **hazır**. Derste sıfırdan kod yazmıyoruz; hazır sayfayı açıp **üretilen HTML'i** tartışıyoruz. Yalnızca Uygulama A'da tek satır, Uygulama C'de tek attribute değiştiriliyor.

Uygulama A — `asp-*` ne yapıyor, kaldırınca ne oluyor?

Süre: 10 dakika · **Blok:** A

Amaç: Tag Helper'ın attribute'ünü sildiğini göstermek ve elle yazılan adresin neden kırıldığını **canlı kanıtlamak**.

Adım 1 — Teşhis kuralını kurun (2 dk)

`/Etiket/Baglanti` sayfasını açın, **Ctrl+U** yapın.

Kaynakta `asp-controller` **yoktur**; yalnızca `href` vardır.

> **Tahtaya yazılacak tek cümle:** *Çalışan Tag Helper kendi attribute'ünü siler.*

Sonra `/Etiket/Eksik` sayfasını açın ve **Ctrl+U** yapın. Burada `asp-for`, `asp-controller` ve `asp-action` **olduğu gibi** duruyor.

Sayfa	Durum kodu	Kaynakta `asp-`
`/Etiket/Baglanti`	200	yok → çalıştı
`/Etiket/Eksik`	**200**	**var** → çalışmadı

İkisi de 200. Bu haftanın en sinsi hatası budur.

Adım 2 — `asp-route-*` nereye yazılıyor? (3 dk)

`/Etiket/Baglanti` sayfasındaki tabloyu okuyun. Her bağlantının ürettiği adres altında yazıyor:

Yazılan	Üretilen
`asp-route-id="3"`	`/Kitap/Detay/3`
`asp-route-kelime="suc"`	`/Kitap/Ara?kelime=suc`

Farkın sebebini `Program.cs`'teki rota şablonundan gösterin: `{controller}/{action}/{id:int?}` — `id` şablonda var, `kelime` yok.

Adım 3 — 🕳️ Rotayı değiştirin (4 dk)

`Program.cs`'te tek satır:

```
// önce
pattern: "{controller=Home}/{action=Index}/{id:int?}"
// sonra
pattern: "site/{controller=Home}/{action=Index}/{id:int?}"
```

Uygulamayı yeniden başlatın, `/site/Etiket/Baglanti` adresini açın.

Beklenen:

Bağlantı	Yeni hâli	Tıklayınca
Tag Helper ile üretilen dördü	`/site/Kitap/...`	**200**
Elle yazılan `href="/Kitap/Detay/3"`	**değişmedi**	**404**

Adım 4 — Geri alın (1 dk)

Rota satırını eski hâline döndürün ve uygulamayı yeniden başlatın. Sonraki uygulamalar normal adreslerle devam edecek.

> ⚠️ **Yedek plan:** Yeniden başlatma uzun sürerse Adım 3 atlanıp doğrudan `/Etiket/Baglanti` sayfasındaki tablo okunabilir; sayfa beklenen sonucu zaten yazıyor.

Uygulama B — İki form, iki HTML

Süre: 11 dakika · **Blok:** B

Amaç: Aynı formun elle ve Tag Helper'la yazılmış hâllerini **üretilen HTML üzerinden** karşılaştırmak.

Adım 1 — İki sekmeyi yan yana açın (1 dk)

- Sekme 1: `/Kitap/Ekle` — Hafta 4, **elle**
- Sekme 2: `/Kitap/EkleTH` — bu hafta, **Tag Helper**

Ekranda ikisi **birbirinin aynısı** görünüyor. Bu önemlidir: fark kullanıcıda değil, **kaynakta**.

Adım 2 — İki kaynağı karşılaştırın (4 dk)

Her iki sekmede **Ctrl+U**. Dört şeye sırayla bakın:

Bakılacak	Elle	Tag Helper
`name`	Biz yazdık	Özellik adından üretildi
`type`	Hepsi `text`	`text`, `text`, `text` … **ama** `Fiyat` için de `text`
`<label>` metni	Elle yazdık	`[Display(Name = "…")]`
Formun son satırı	—	`__RequestVerificationToken`

Son satır Blok C'nin kapısıdır; burada yalnızca **varlığı** gösterilir.

Adım 3 — `asp-for` galerisini açın (3 dk)

`/Etiket` sayfası her tip için üretilen HTML'i yan yana basıyor. Üç satırı vurgulayın:

1. `int` → `type="number"`, `decimal` → `type="text"`. Sebebi sayfada yazıyor: `120,50` virgüllü, `type="number"` yalnızca nokta kabul eder.
2. `bool` → `checkbox` + gizli `false` alanı. Hafta 4'te "işaretlenmemiş kutu hiç gelmez" demiştik; açık burada kapanıyor.
3. `string` → `data-val-required` var, `string?` → yok. Farkı yapan nullable reference types.

Adım 4 — Yazım hatasını yapın (3 dk)

`Views/Kitap/EkleTH.cshtml` dosyasında:

```
<input asp-for="Baslik" size="40" />
@* ↓ *@
<input asp-for="Balsik" size="40" />
```

`dotnet build` çalıştırın.

```
error CS1061: 'KitapEkleVM' does not contain a definition for 'Balsik'
```

> **Haftanın cümlesi burada söylenir:** Hafta 4'te aynı hata sayfayı sessizce boş bırakıyordu. Şimdi **proje derlenmiyor**.

Hatayı geri alın.

> ⚠️ **Yedek plan:** Derleme uzun sürerse hata metni slaytta da duruyor; `dotnet build` beklenmeden devam edilebilir.

Uygulama C — Token'ı sökmek

Süre: 12 dakika · **Blok:** C

Amaç: Token'ın kim tarafından, ne zaman üretildiğini ve kontrol edilmediğinde ne işe yaramadığını göstermek.

Adım 1 — Üç formu karşılaştırın (3 dk)

`/Etiket/Csrf` sayfasını açın. Üç form var; ikisi **aynı action'a** gidiyor.

Form	Token	Gönderince
Tag Helper + POST	**var**	Çalışır
Tag Helper + GET	**yok**	(GET; doğrulama aranmaz)
Elle yazılmış POST	**yok**	**400**

Üçüncü düğmeye basın. **Boş bir sayfa** gelir.

> Üç gözlem yüksek sesle söylenir: ekranda açıklama **yok**, exception **yok**, action'ın içi **hiç çalışmadı**.

Adım 2 — Formu Tag Helper'dan çıkarın (4 dk)

`Views/Kitap/EkleTH.cshtml` dosyasında form etiketini elle yazılmış hâle çevirin:

```
@* önce *@
<form asp-controller="Kitap" asp-action="EkleTH" method="post">
@* sonra *@
<form action="/Kitap/EkleTH" method="post">
```

Sayfayı yenileyin, **Ctrl+U** yapın: token **kayboldu**.

Formu doldurup gönderin: **400**.

Çıkarılacak sonuç: adresi elle yazmak yalnızca bağlantıyı kırmaz; **güvenlik alanını da götürür**. İki tuzak, tek sebep.

Değişikliği geri alın.

Adım 3 — Doğrulamayı kaldırın (3 dk)

`KitapController.cs` içinde:

```
[HttpPost]
[ValidateAntiForgeryToken]   // ← bu satırı yorum yapın
public IActionResult EkleTH(KitapEkleVM girdi)
```

Şimdi **elle yazılmış** form da çalışır.

Çıkarılacak sonuç: token'ın **üretilmesi** tek başına hiçbir şey yapmaz. Üretmek Tag Helper'ın işi, **kontrol etmek sizin işiniz**.

Satırı geri açın.

Adım 4 — `method="get"` deneyin (2 dk)

`/Etiket/Csrf` sayfasındaki ikinci form `method="get"`. Kaynakta token yok.

Soruyu kapatın: **neden?** Çünkü CSRF, kullanıcıya istemediği bir **değişikliği** yaptırma saldırısıdır. GET bir okuma isteğidir.

> Tersini de söyleyin: veri **değiştiren** bir işlemi GET'e koyarsanız bu korumayı kendi elinizle kapatırsınız. `<a href="/Kitap/Sil/3">` bu yüzden tehlikelidir.

🏠 Ders Sonrası Öğrenci Görevi

1. Hafta 4'te **elle** yazdığınız formu, bu kez **Tag Helper'larla** yeniden yazın. Eski dosyayı **silmeyin**; ikisi yan yana dursun.
2. İki formun ürettiği HTML'i **Ctrl+U** ile alıp yan yana koyun ve **farkları işaretleyin**.

3. `name`, `id` ve `type` değerlerinin Tag Helper'lı sürümde **nereden geldiğini** her biri için tek cümleyle yazın.
4. Modelinize `[Display(Name = "…")]` ekleyip `<label asp-for>` çıktısının değiştiğini gösterin.
5. Bir `asp-for` değerini **bilerek yanlış** yazın. Derleyicinin verdiği hatayı ekran görüntüsüyle kaydedin ve Hafta 4'te aynı hatanın ne yaptığıyla **karşılaştırın**.
6. POST action'ınıza `[ValidateAntiForgeryToken]` ekleyin. Sonra formu **elle yazılmış** hâle çevirip gönderin; aldığınız **durum kodunu** yazın.
7. `Program.cs`'teki rota şablonuna bir önek ekleyin. Hangi bağlantılar taşındı, hangileri kırıldı? Ekran görüntüsüyle gösterin.

Teslim: proje klasörü + iki HTML çıktısının **işaretlenmiş** karşılaştırması + 5., 6. ve 7. maddelerin ekran görüntüleri + şu soruya **iki cümlelik** yazılı cevap: "*Tag Helper bir kısaltma mı, yoksa başka bir şey mi?*"

✅ Hocaya Kontrol Listesi

- [ ] Kaynak penceresinin (Ctrl+U) yazı tipi **en arka sıradan okunacak** kadar büyük
- [ ] `/Kitap/Ekle` ve `/Kitap/EkleTH` iki ayrı sekmede, **geçiş hızlı**
- [ ] `Program.cs` ve `EkleTH.cshtml` editörde **açık sekmeler** olarak hazır
- [ ] Uygulama A Adım 4 (rotayı geri alma) **atlanmadı**
- [ ] Uygulama C'de **iki geri alma** da yapıldı (form etiketi, `[ValidateAntiForgeryToken]`)
- [ ] Ders sonunda `git status` temiz — derste yapılan bozmalar geri alındı

⚠️ Yedek Planların Özeti

Sorun	Yedek
`dotnet run` yeniden başlatma yavaş	Uygulama A Adım 3 atlanır; sayfadaki tablo okunur
`dotnet build` yavaş	Uygulama B Adım 4'ün hata metni slaytta duruyor
Ağ yok (3B sahne açılmıyor)	Sahne kendini kapatır, 2B şema devreye girer
Proje hiç açılmıyor	Bütün çıktılar ders notunda tablo hâlinde var; slaytlardan anlatılır